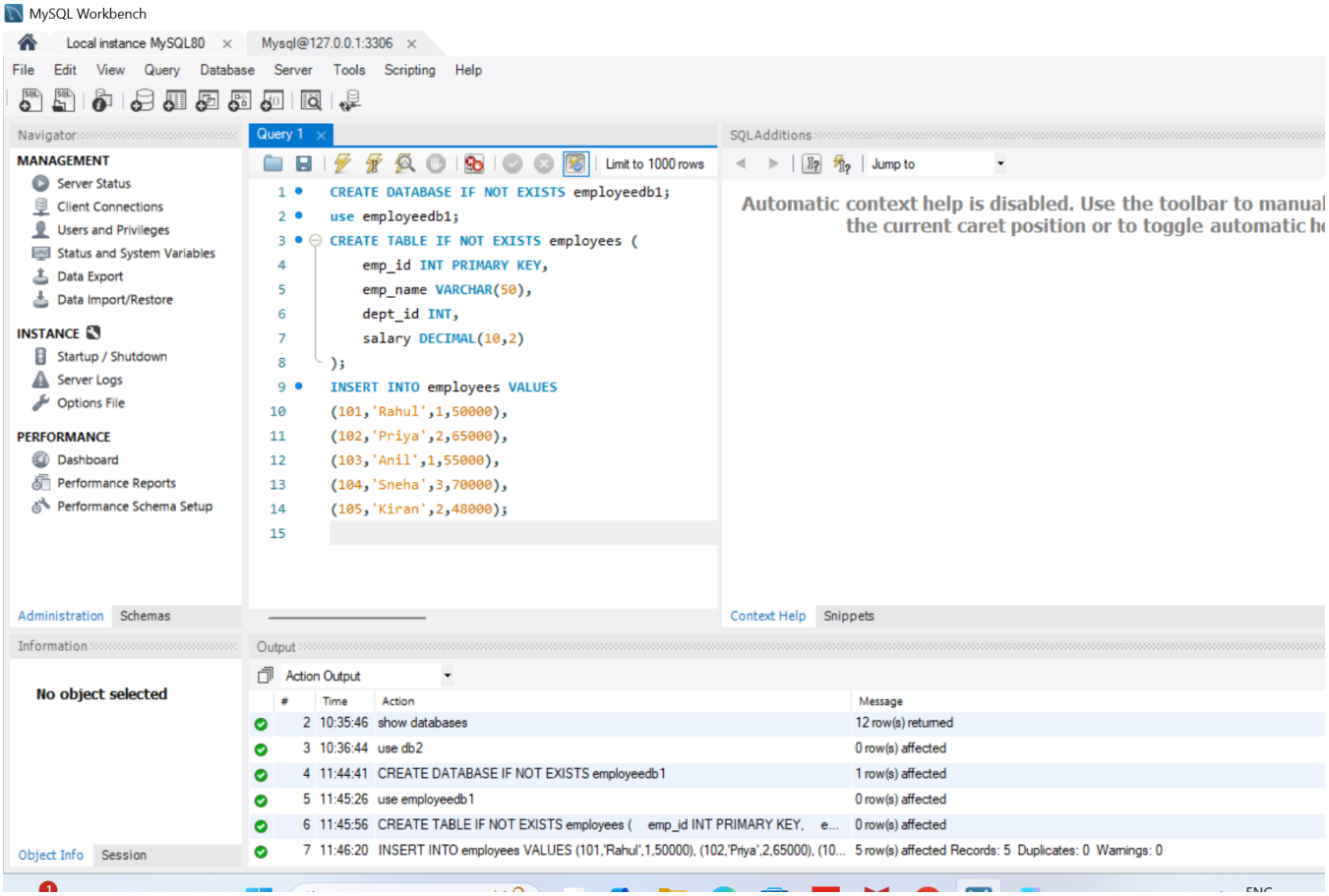
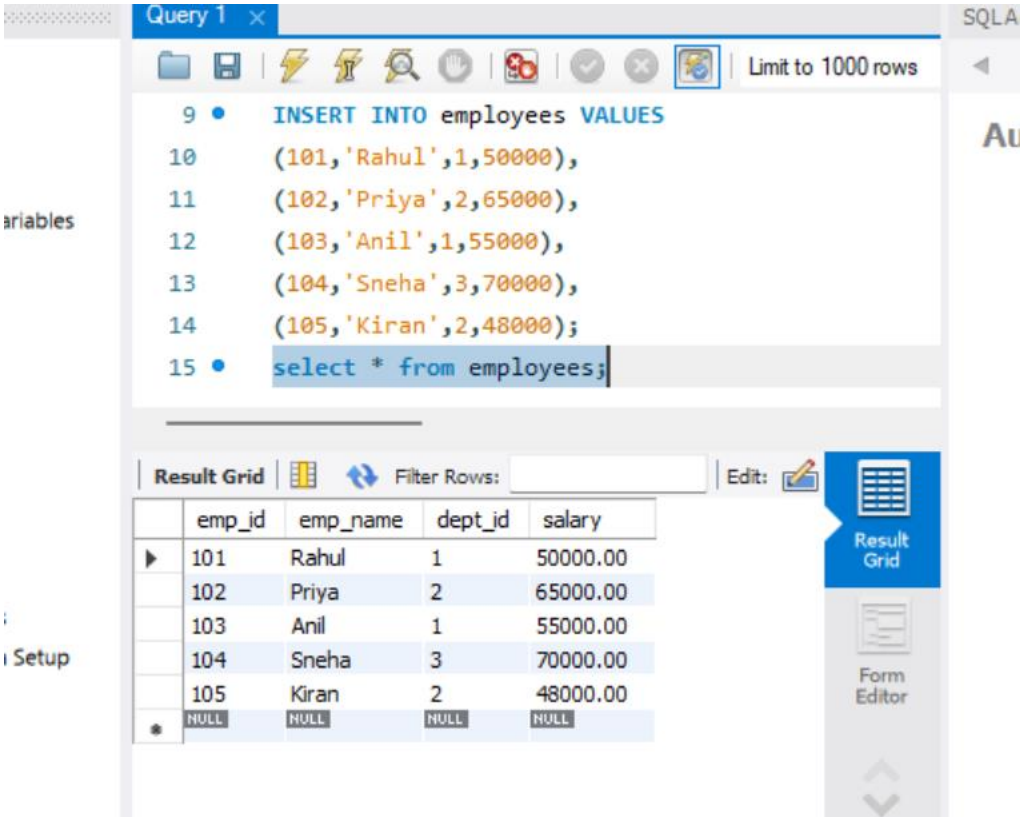
MYSQL STORED PROCEDURES

Examples and Practice Exercises for MySQL Workbench

1. Setup: Employees Table

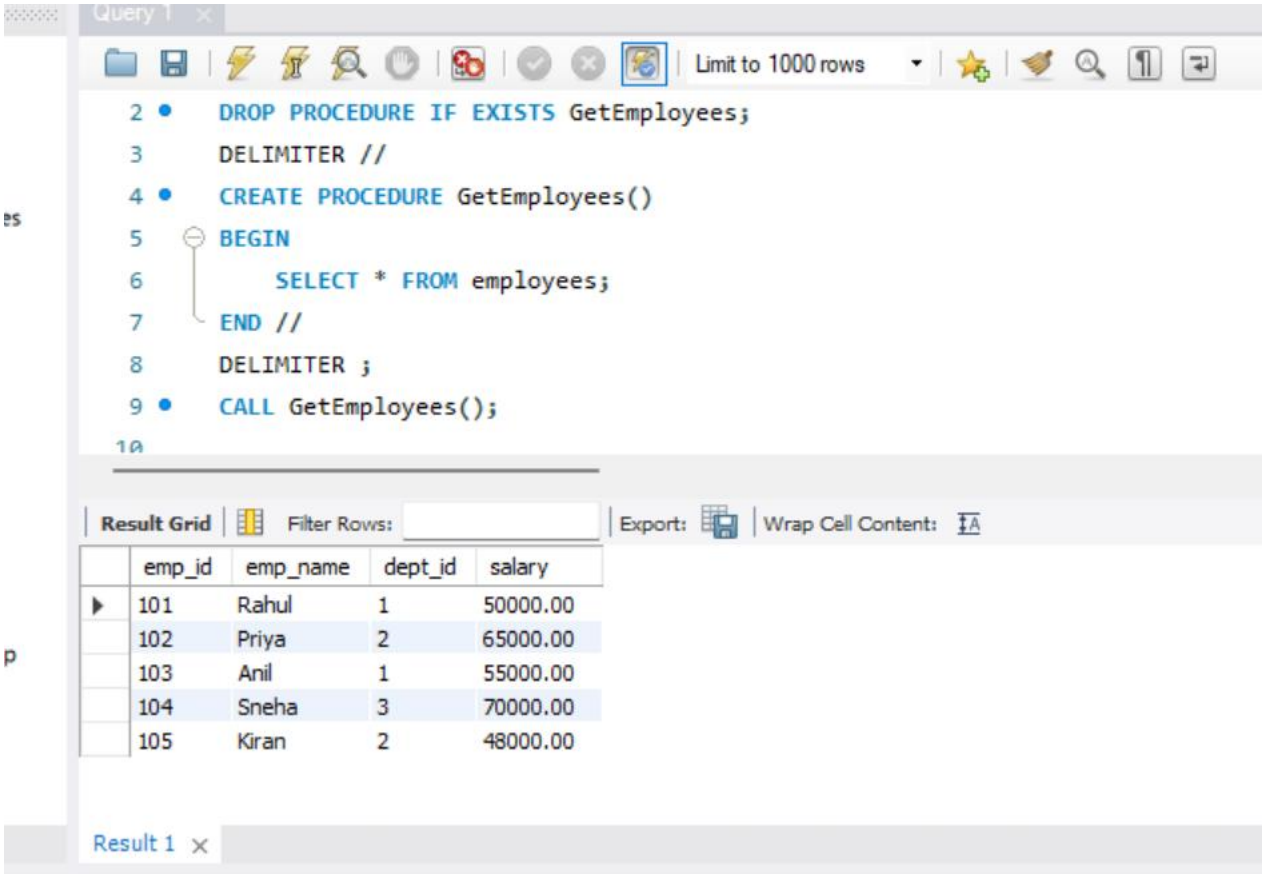


2. Verify Data



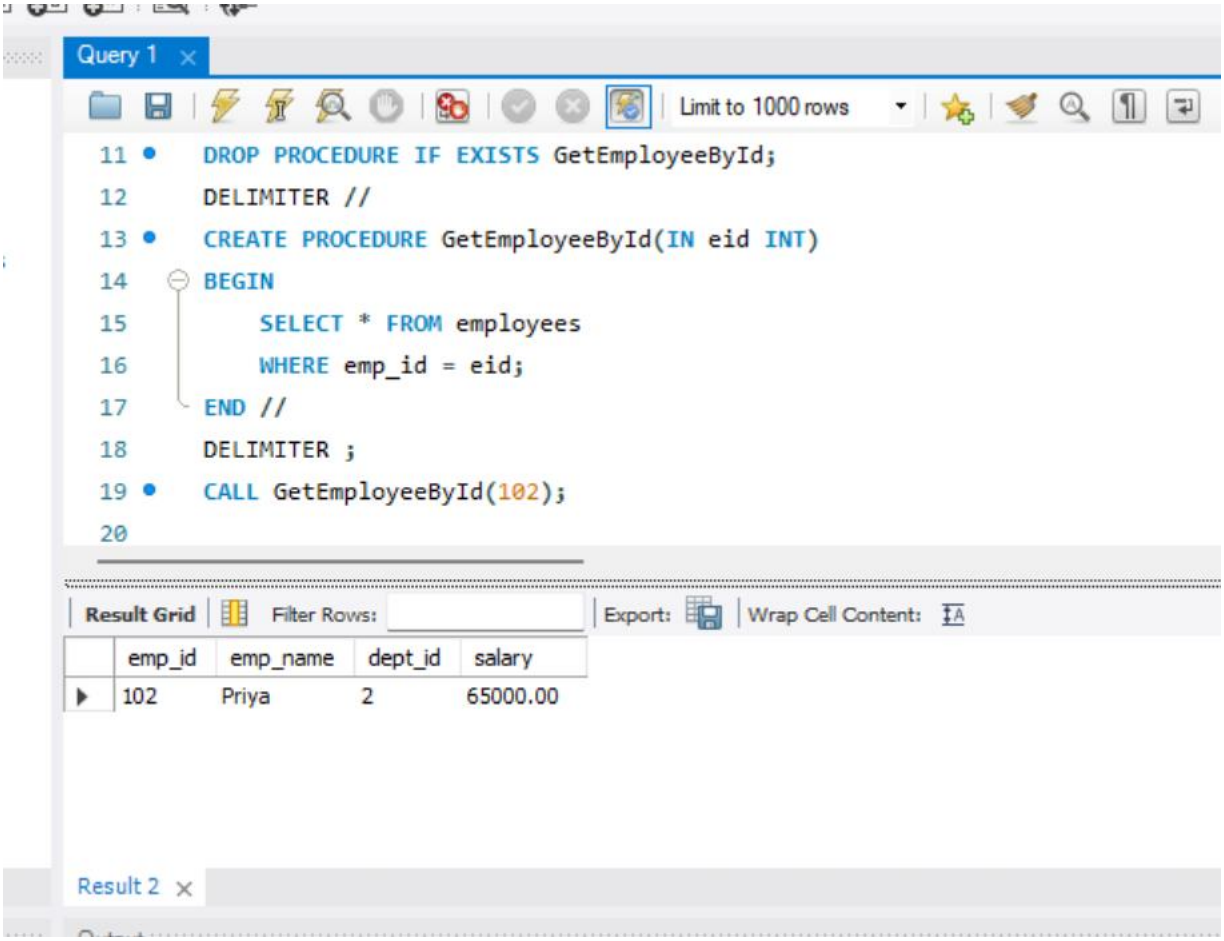
Procedure 1 – Display All Employees

Question: Create a procedure that displays all employees



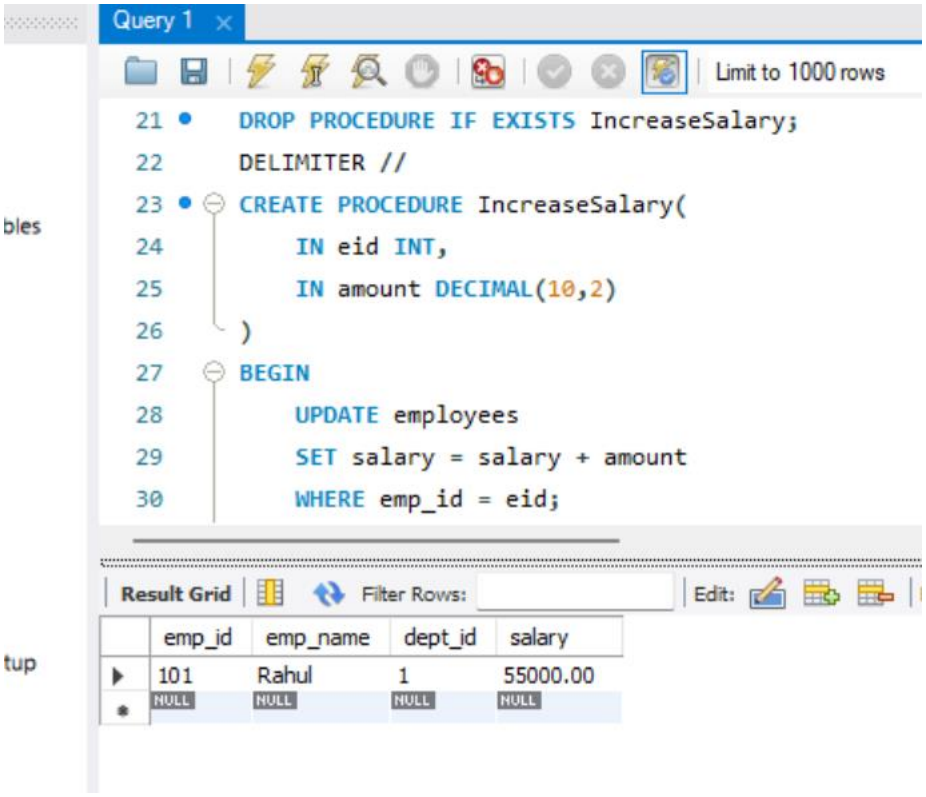
Procedure 2 – IN Parameter

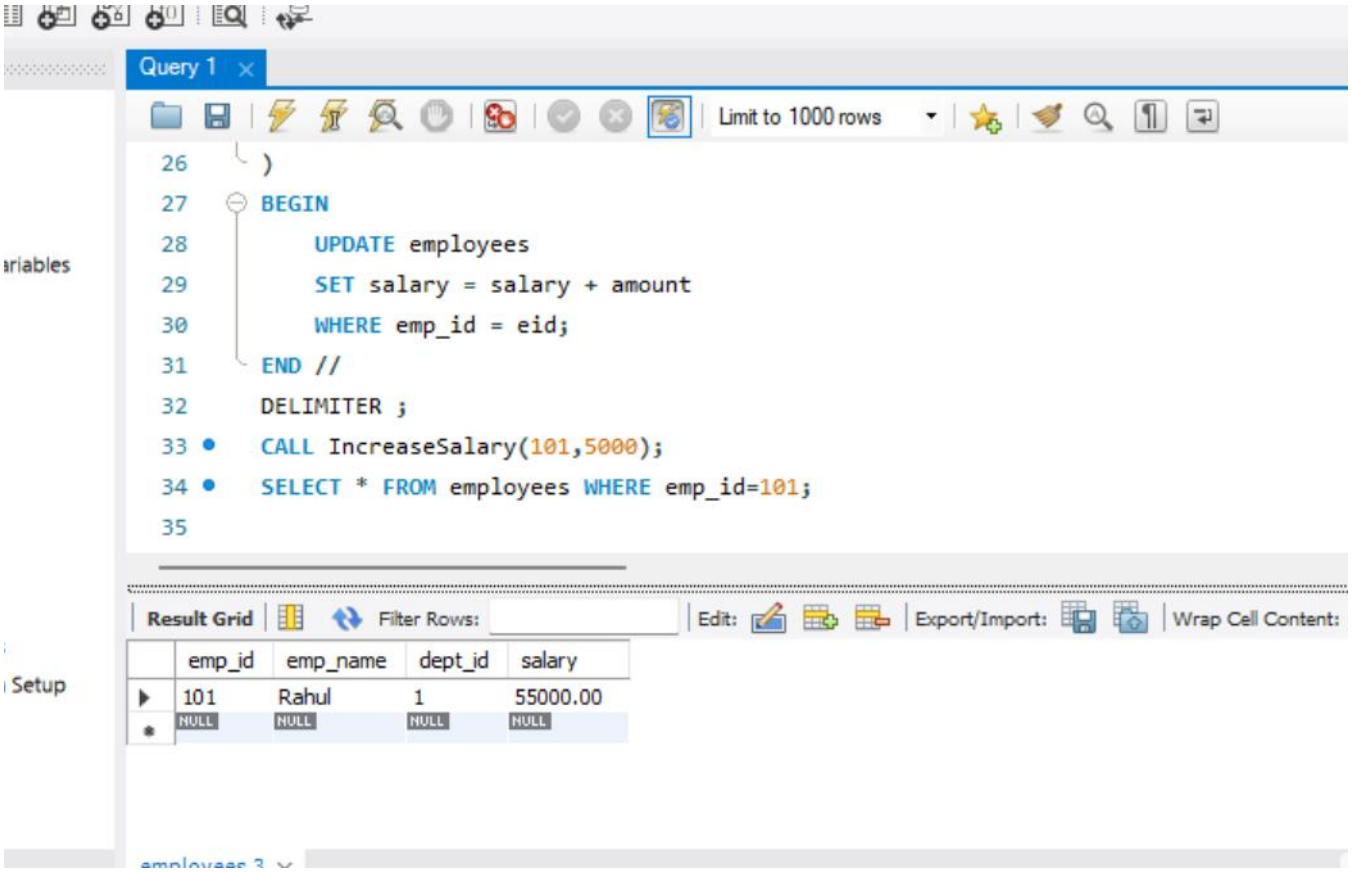
Question: Create a procedure that accepts an employee ID and displays that employee's details.



Procedure 3 – Update Salary

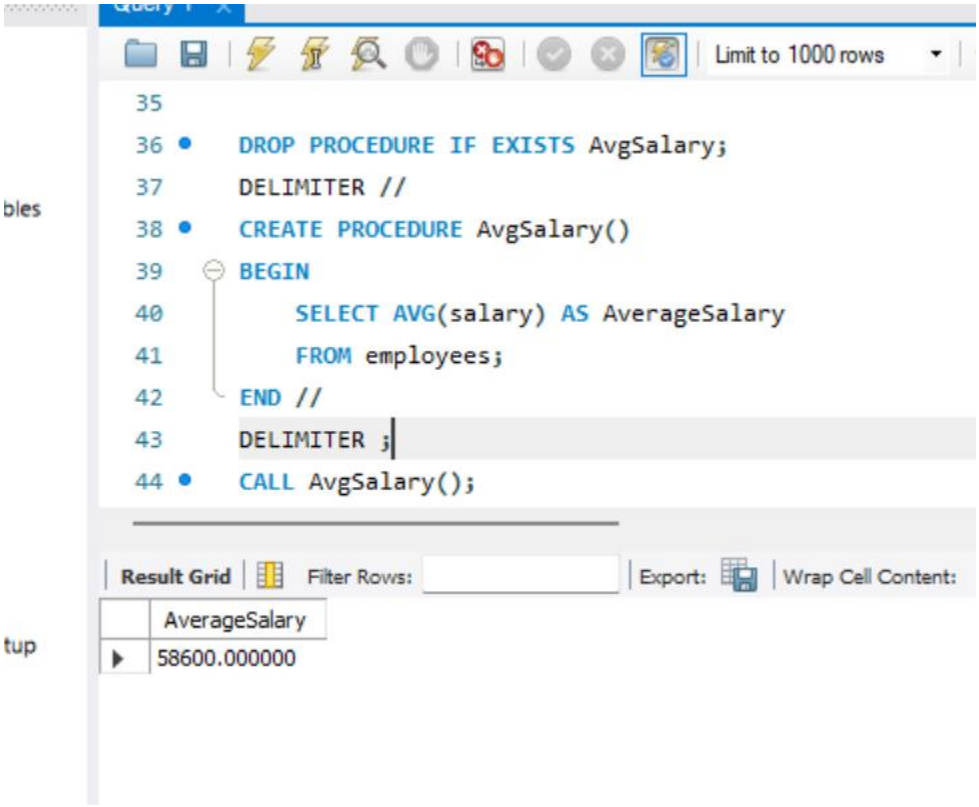
Question: Create a procedure that accepts employee ID and amount, then increases the employee's salary.





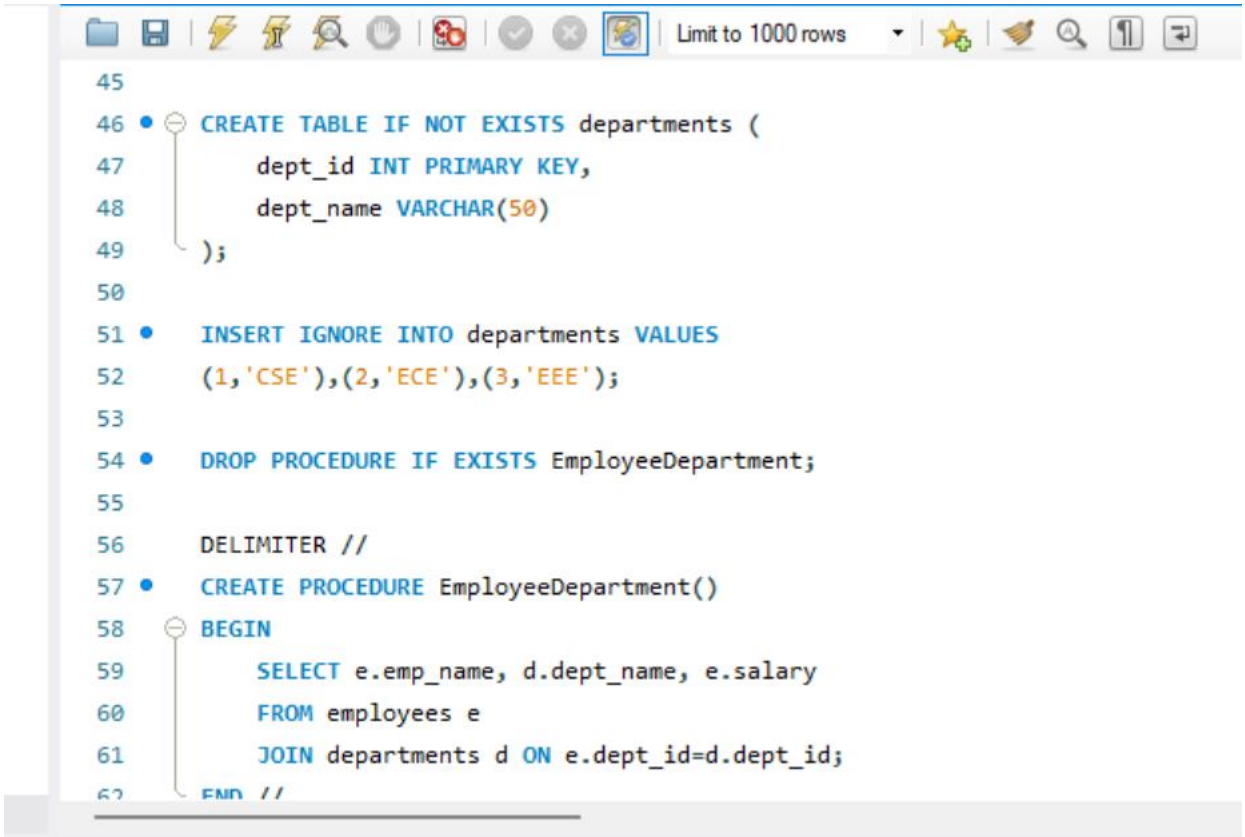
Procedure 4 – Aggregate Function

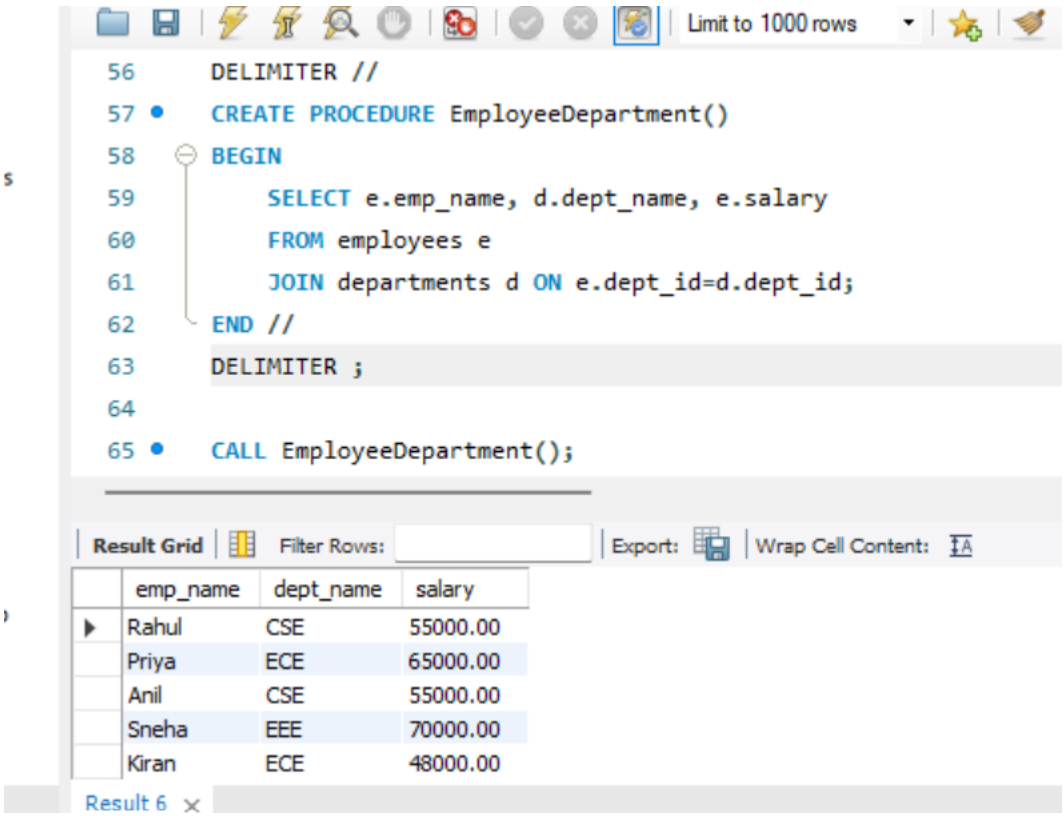
Question: Create a procedure to calculate the average salary of all employees.



Procedure 5 – JOIN

Question: Create a procedure to display employee name, department name and salary. Run the department setup first.





```
56    DELIMITER //
57    CREATE PROCEDURE EmployeeDepartment()
58    BEGIN
59        SELECT e.emp_name, d.dept_name, e.salary
60        FROM employees e
61        JOIN departments d ON e.dept_id=d.dept_id;
62    END //
63    DELIMITER ;
64
65    CALL EmployeeDepartment();
```

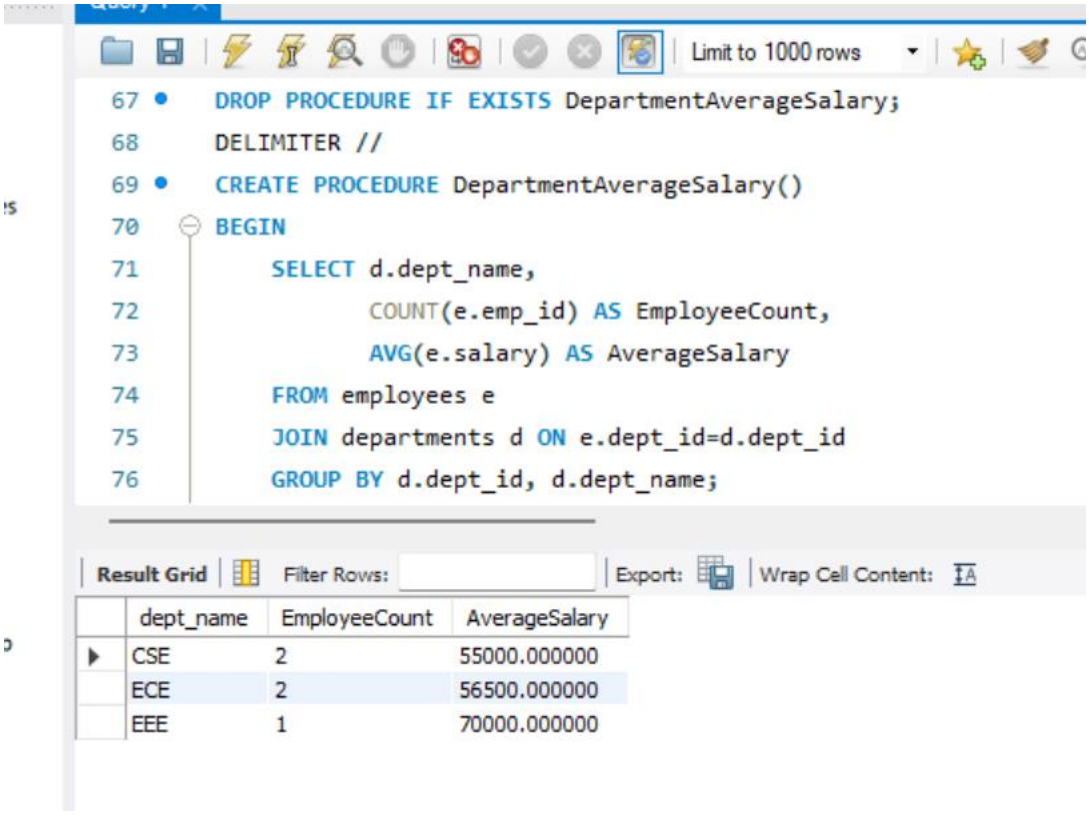
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_name	dept_name	salary
Rahul	CSE	55000.00
Priya	ECE	65000.00
Anil	CSE	55000.00
Sneha	EEE	70000.00
Kiran	ECE	48000.00

Result 6

Procedure 6 – GROUP BY and AVG

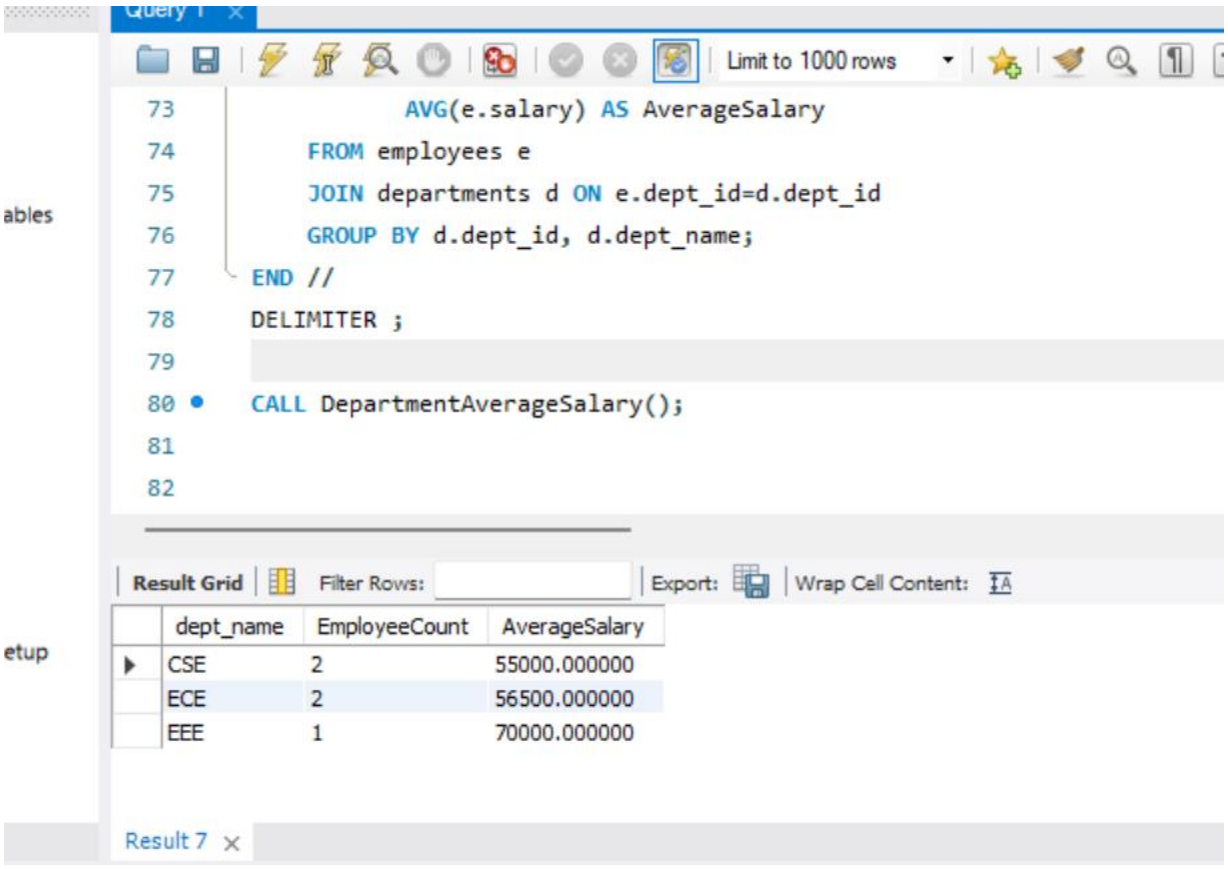
Question: Create a procedure to display department-wise employee count and average salary.



```
67    DROP PROCEDURE IF EXISTS DepartmentAverageSalary;
68    DELIMITER //
69    CREATE PROCEDURE DepartmentAverageSalary()
70    BEGIN
71        SELECT d.dept_name,
72               COUNT(e.emp_id) AS EmployeeCount,
73               AVG(e.salary) AS AverageSalary
74        FROM employees e
75        JOIN departments d ON e.dept_id=d.dept_id
76        GROUP BY d.dept_id, d.dept_name;
```

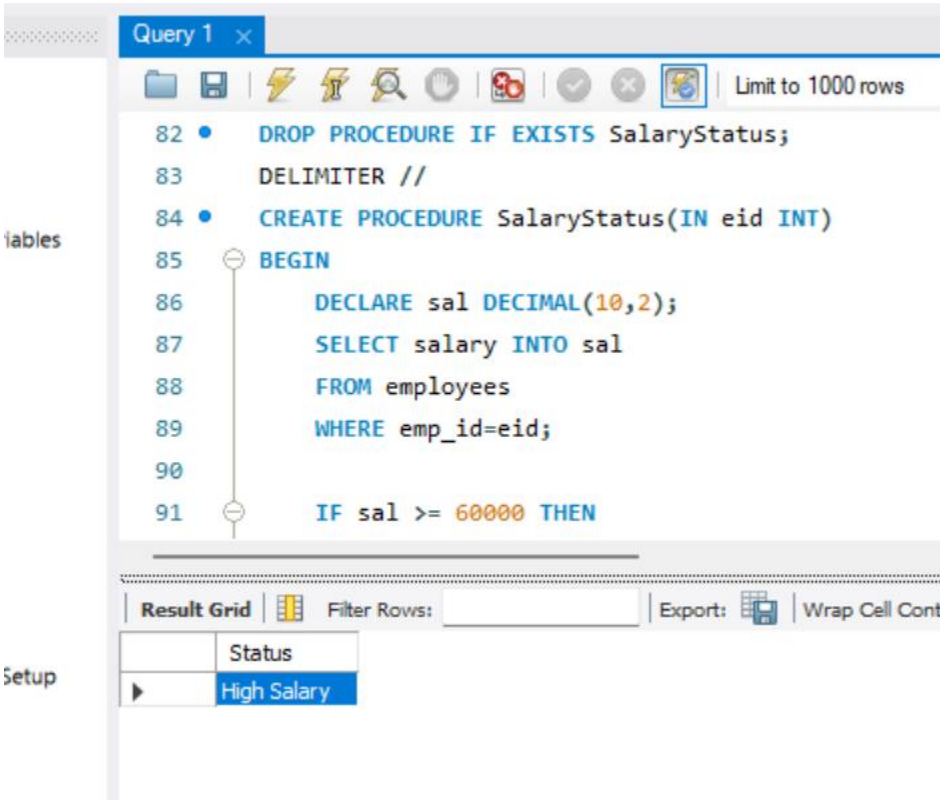
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

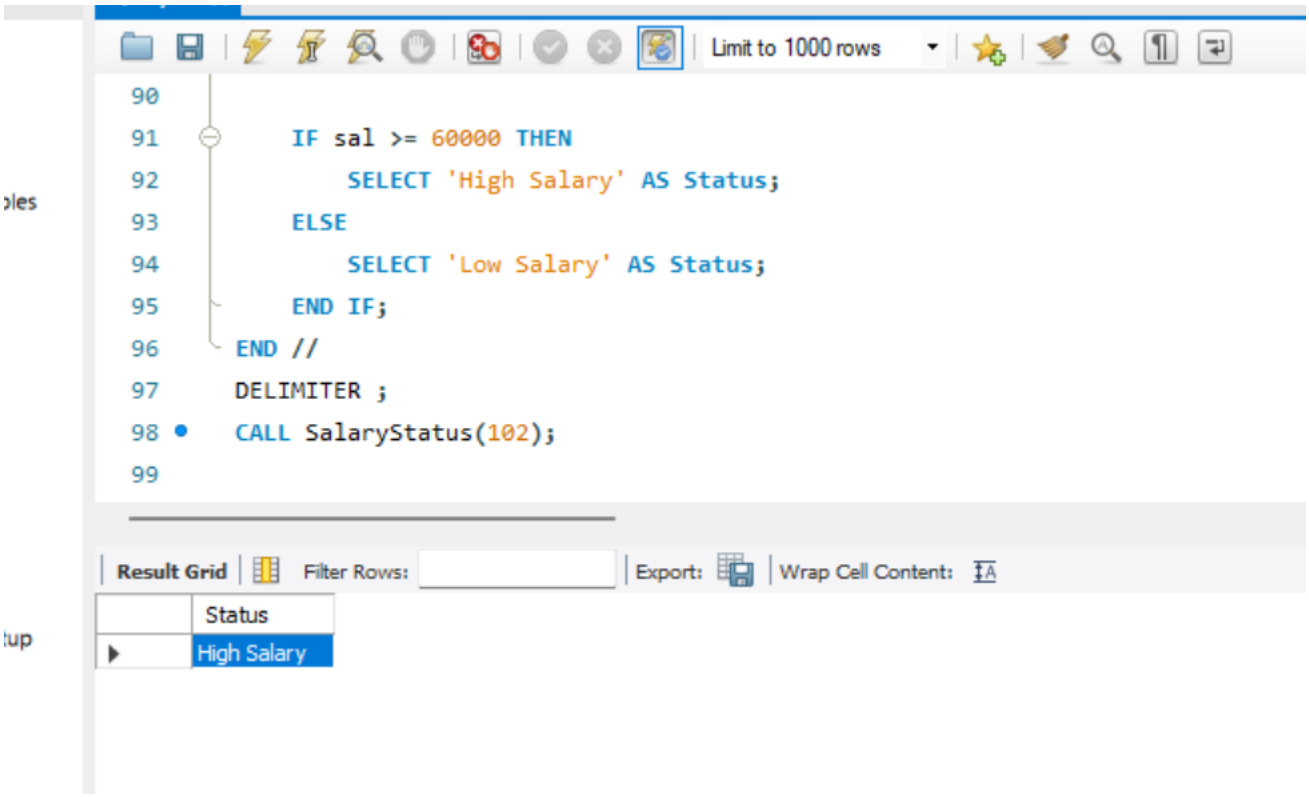
dept_name	EmployeeCount	AverageSalary
CSE	2	55000.000000
ECE	2	56500.000000
EEE	1	70000.000000


Query 1
Limit to 1000 rows
AVG(e.salary) AS AverageSalary
FROM employees e
JOIN departments d ON e.dept_id=d.dept_id
GROUP BY d.dept_id, d.dept_name;
END //
DELIMITER ;
CALL DepartmentAverageSalary();
Result Grid
Filter Rows:
Export:
Wrap Cell Content:
dept_name
EmployeeCount
AverageSalary
CSE
2
55000.000000
ECE
2
56500.000000
EEE
1
70000.000000
Result 7

Procedure 7 – IF...ELSE

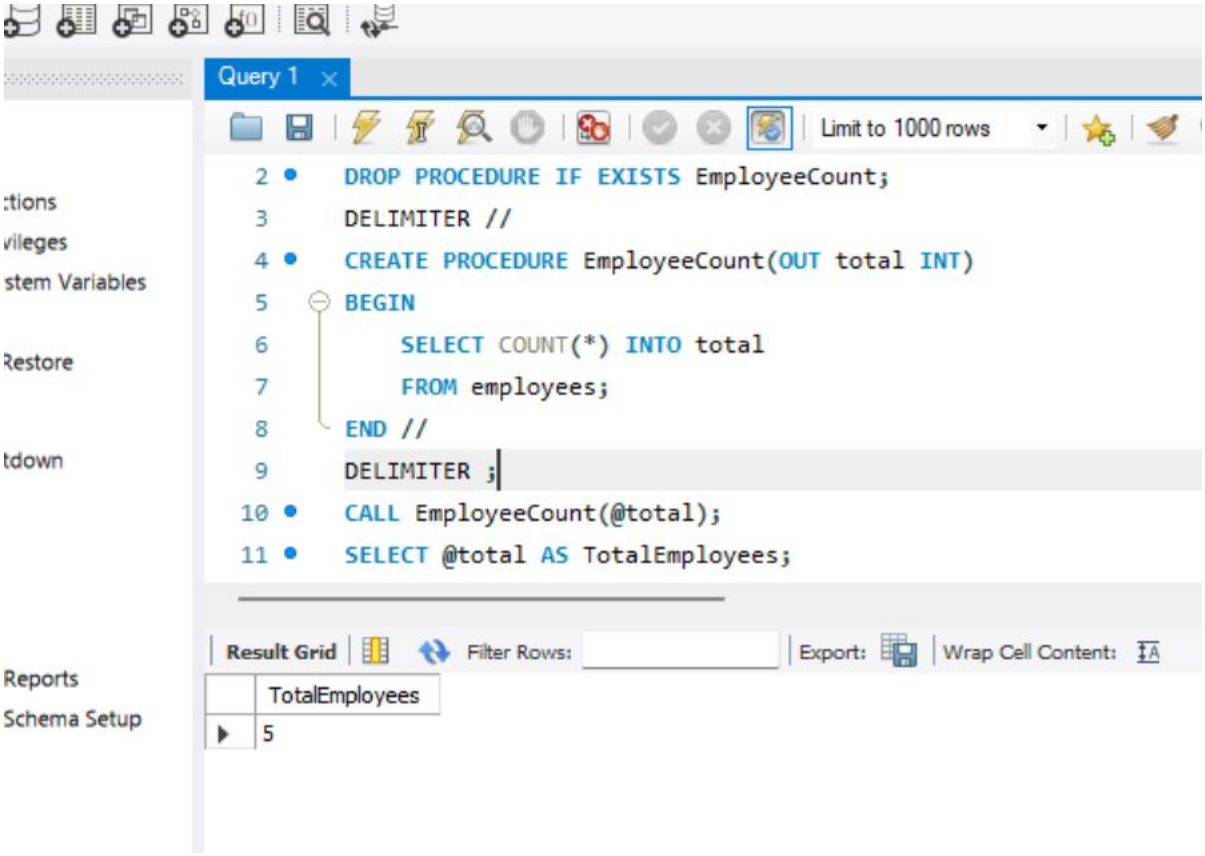
Question: Display High Salary if salary is at least 60000; otherwise display Low Salary.





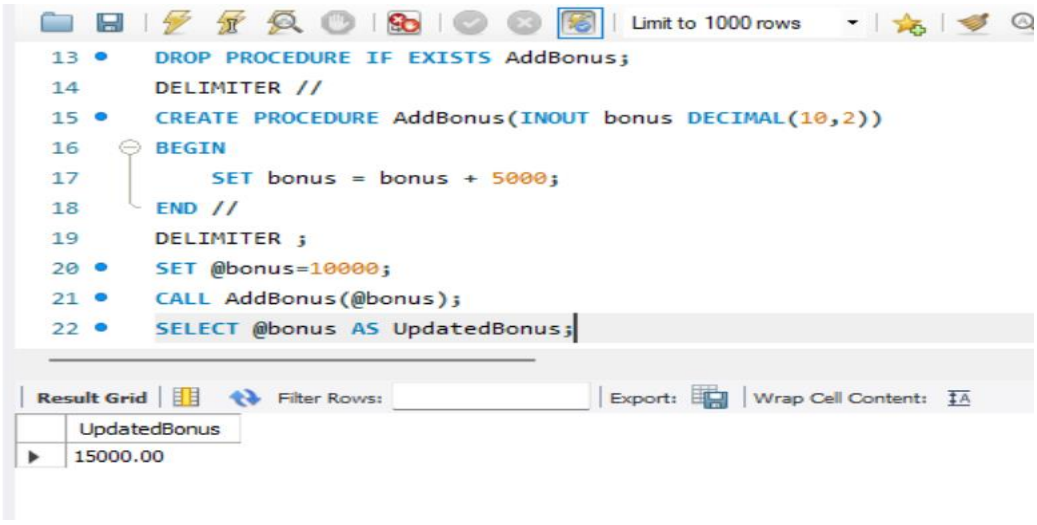
Procedure 8 – OUT Parameter

Question: Return the total number of employees using an OUT parameter.



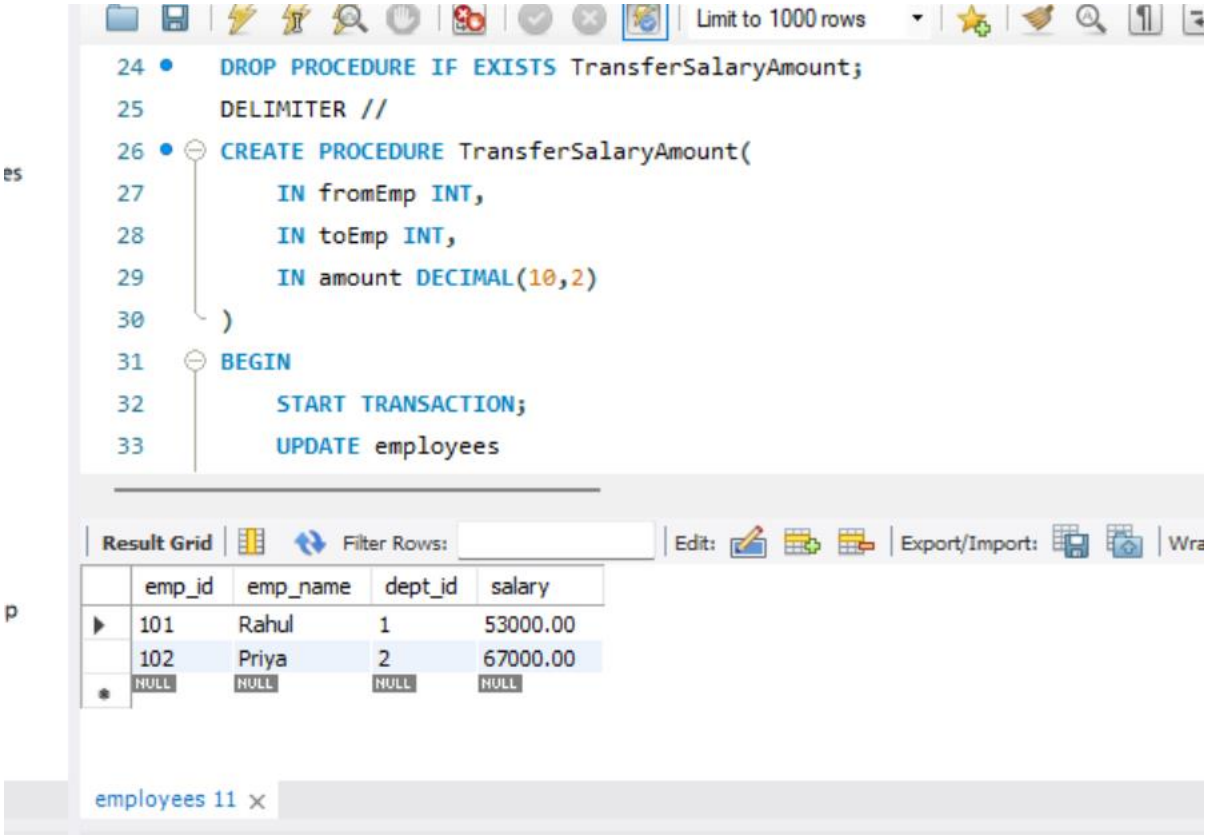
Procedure 9 – INOUT Parameter

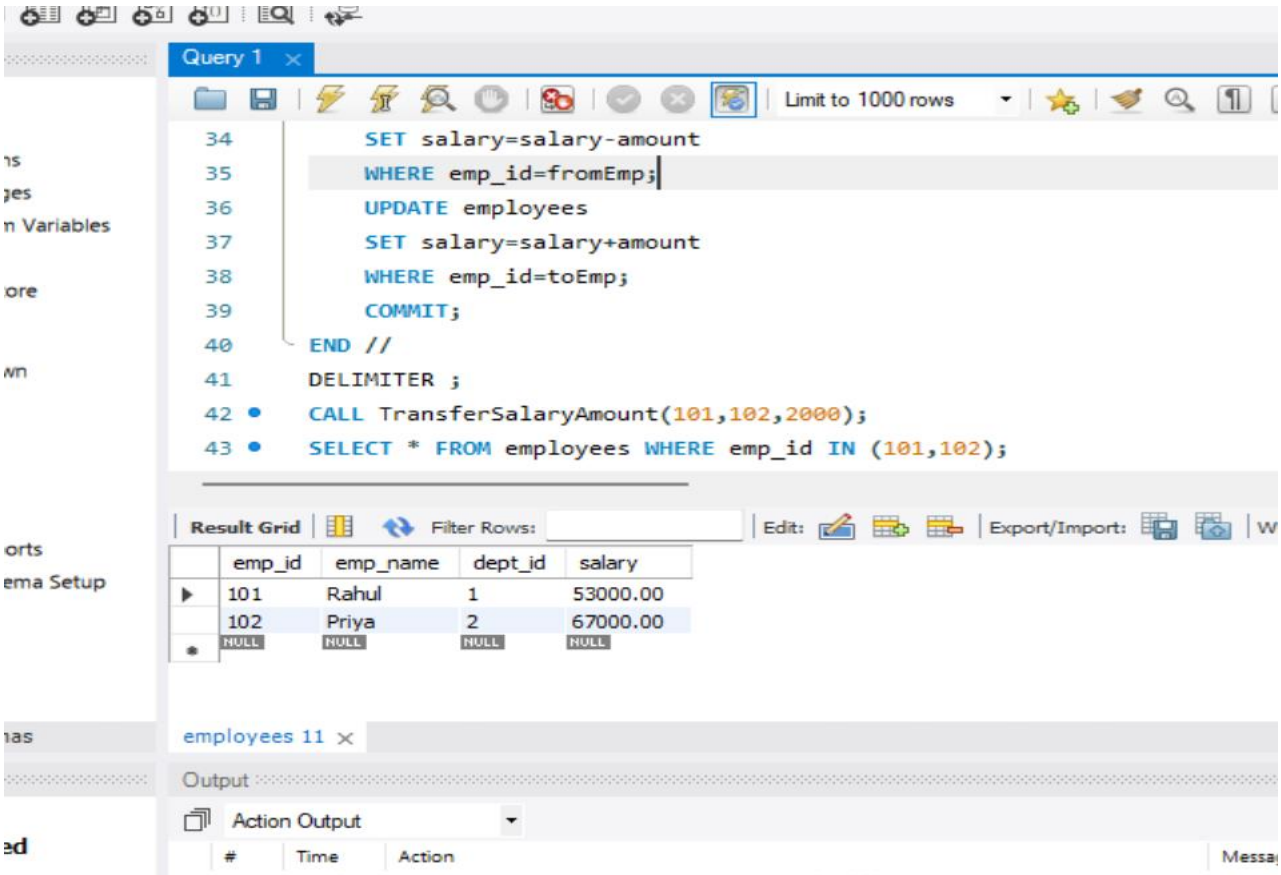
Question: Receive a bonus through an INOUT parameter and add 5000 to it.



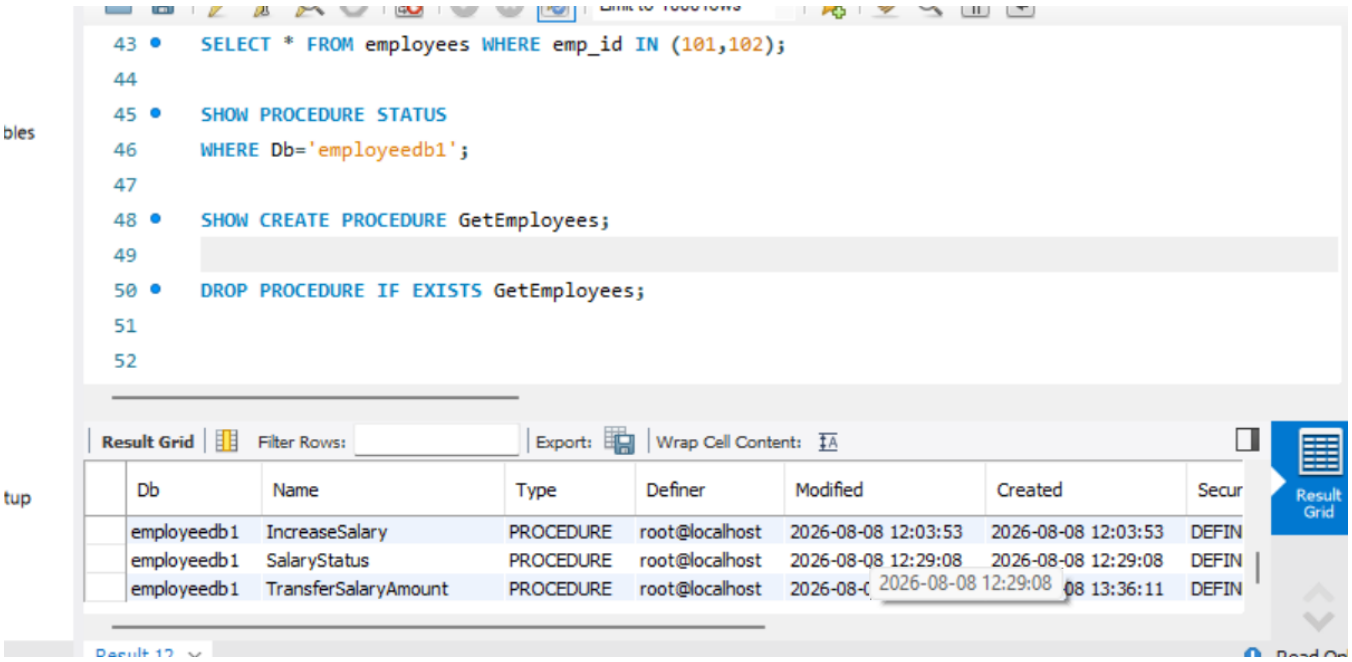
Procedure 10 – Transaction

Question: Transfer an amount from one employee salary record to another using START TRANSACTION and COMMIT.


Limit to 1000 rows
24 DROP PROCEDURE IF EXISTS TransferSalaryAmount;
25 DELIMITER //
26 CREATE PROCEDURE TransferSalaryAmount(
27 IN fromEmp INT,
28 IN toEmp INT,
29 IN amount DECIMAL(10,2)
30)
31 BEGIN
32 START TRANSACTION;
33 UPDATE employees
Result Grid
Filter Rows:
Edit:
Export/Import:
emp_id emp_name dept_id salary
101 Rahul 1 53000.00
102 Priya 2 67000.00
NULL NULL NULL NULL
employees 11


Query 1
Limit to 1000 rows
34 SET salary=salary-amount
35 WHERE emp_id=fromEmp;
36 UPDATE employees
37 SET salary=salary+amount
38 WHERE emp_id=toEmp;
39 COMMIT;
40 END //
41 DELIMITER ;
42 CALL TransferSalaryAmount(101,102,2000);
43 SELECT * FROM employees WHERE emp_id IN (101,102);
Result Grid
Filter Rows:
Edit:
Export/Import:
emp_id emp_name dept_id salary
101 Rahul 1 53000.00
102 Priya 2 67000.00
NULL NULL NULL NULL
employees 11
Output
Action Output
Time Action

4. Useful Procedure Commands



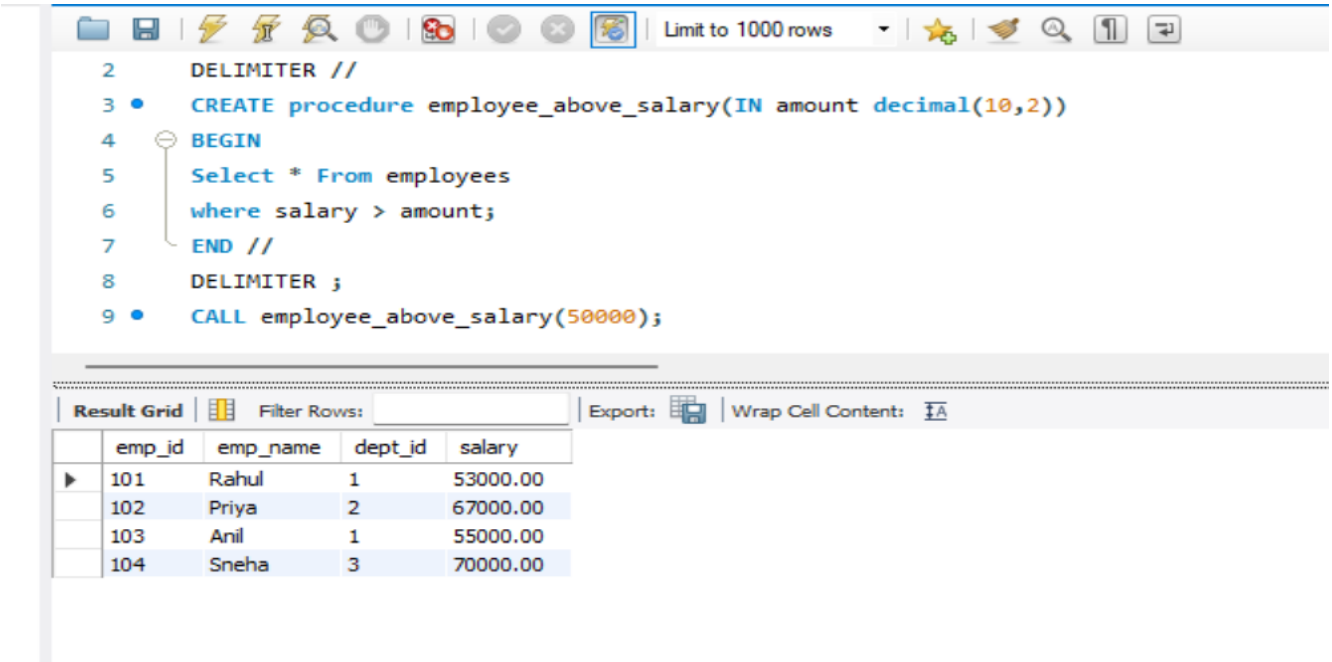
```
43  SELECT * FROM employees WHERE emp_id IN (101,102);
44
45  SHOW PROCEDURE STATUS
46  WHERE Db='employeedb1';
47
48  SHOW CREATE PROCEDURE GetEmployees;
49
50  DROP PROCEDURE IF EXISTS GetEmployees;
51
52
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Db	Name	Type	Definer	Modified	Created	Secur
employeedb1	IncreaseSalary	PROCEDURE	root@localhost	2026-08-08 12:03:53	2026-08-08 12:03:53	DEFIN
employeedb1	SalaryStatus	PROCEDURE	root@localhost	2026-08-08 12:29:08	2026-08-08 12:29:08	DEFIN
employeedb1	TransferSalaryAmount	PROCEDURE	root@localhost	2026-08-(	08 13:36:11	DEFIN

6. Student Practice Tasks

1. Create a procedure to display employees whose salary is greater than a given amount.



```
2   DELIMITER //
3   CREATE procedure employee_above_salary(IN amount decimal(10,2))
4   BEGIN
5   Select * From employees
6   where salary > amount;
7   END //
8   DELIMITER ;
9   CALL employee_above_salary(50000);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	dept_id	salary
101	Rahul	1	53000.00
102	Priya	2	67000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00

2. Create a procedure to update an employee's department using IN parameters.

```
DELIMITER //
CREATE PROCEDURE update_department(
    IN p_emp_id INT,
    IN p_dept_id INT
)
BEGIN
    UPDATE employees
    SET dept_id = p_dept_id
    WHERE emp_id = p_emp_id;
END //
DELIMITER ;
CALL update_department(101,3);
select * from employees;
```

```
34  CALL update_department(101,3);
35  select * from employees;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

emp_id	emp_name	dept_id	salary
101	Rahul	3	53000.00
102	Priya	2	67000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00
105	Kiran	2	48000.00
NULL	NULL	NULL	NULL

3. Create a procedure to return the maximum salary using an OUT parameter.

```
40
41
42
43  DELIMITER //
44  CREATE PROCEDURE max_salary(OUT max_sal DECIMAL(10,2))
45  BEGIN
46      SELECT MAX(salary)
47      INTO max_sal
48      FROM employees;
49  END //
50  DELIMITER ;
51  CALL max_salary(@max);
52  SELECT @max;
```

Output

Action Output

```
43      DELIMITER ;
44      CALL max_salary(@max);
45      SELECT @max;
```

Result Grid | Filter Rows: | Export:

	@max
▶	70000.00

4. Create a procedure to return the employee count of a specified department.

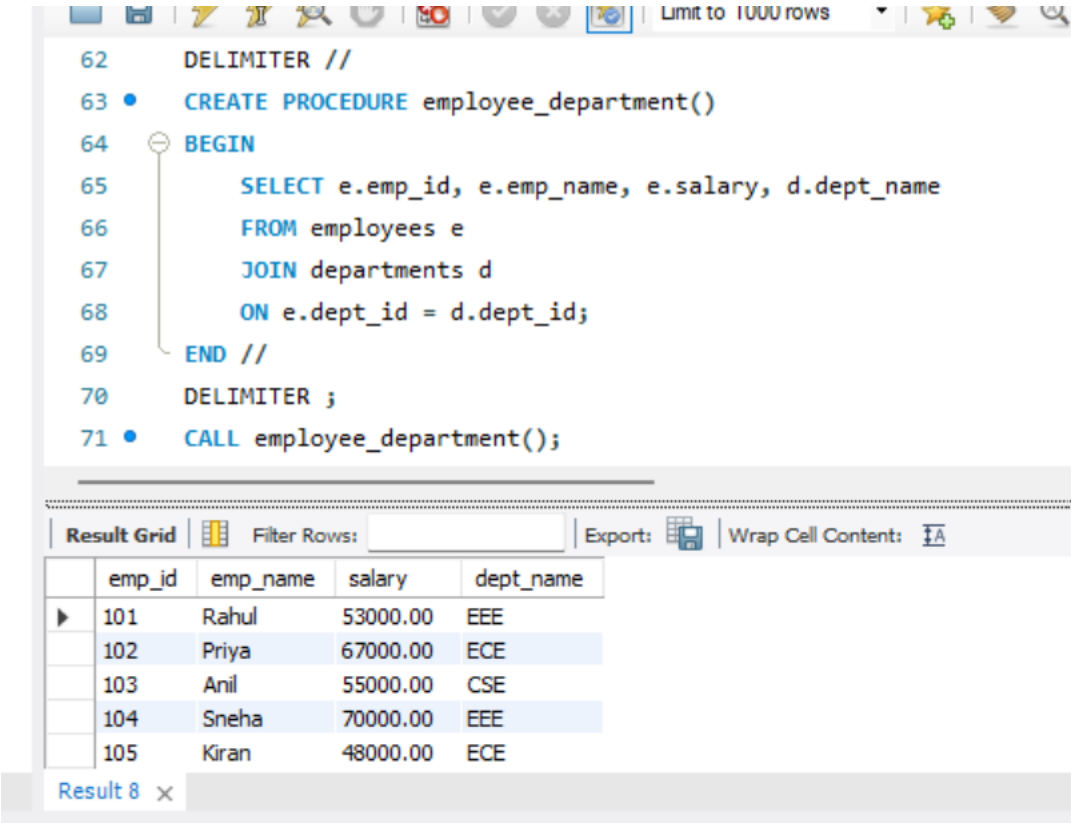
```
46
47      DELIMITER //
48      CREATE PROCEDURE employee_count(
49          IN p_dept_id INT,
50          OUT emp_count INT
51      )
52      BEGIN
53          SELECT COUNT(*)
54          INTO emp_count
55          FROM employees
56          WHERE dept_id = p_dept_id;
57      END //
58      DELIMITER ;
59      CALL employee_count(1, @count);
60      SELECT @count;
```

```
54          INTO emp_count
55          FROM employees
56          WHERE dept_id = p_dept_id;
57      END //
58      DELIMITER ;
59      CALL employee_count(1, @count);
60      SELECT @count;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	@count
▶	1

5. Create a procedure using JOIN to display employee and department details.



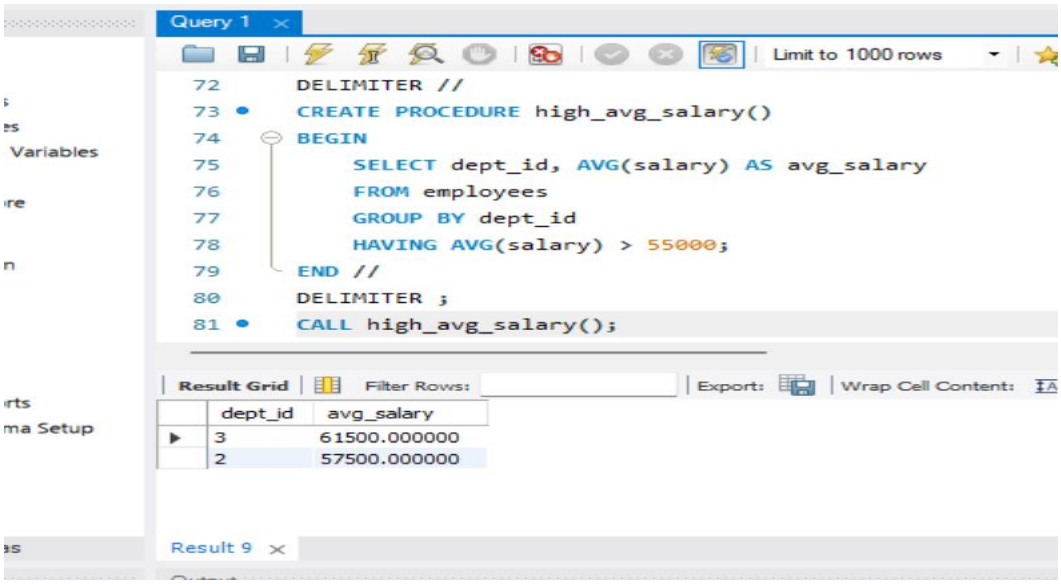
```
62    DELIMITER //
63    CREATE PROCEDURE employee_department()
64    BEGIN
65        SELECT e.emp_id, e.emp_name, e.salary, d.dept_name
66        FROM employees e
67        JOIN departments d
68        ON e.dept_id = d.dept_id;
69    END //
70    DELIMITER ;
71    CALL employee_department();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

emp_id	emp_name	salary	dept_name
101	Rahul	53000.00	EEE
102	Priya	67000.00	ECE
103	Anil	55000.00	CSE
104	Sneha	70000.00	EEE
105	Kiran	48000.00	ECE

Result 8

6. Create a procedure to display departments whose average salary is greater than 55000.



Query 1

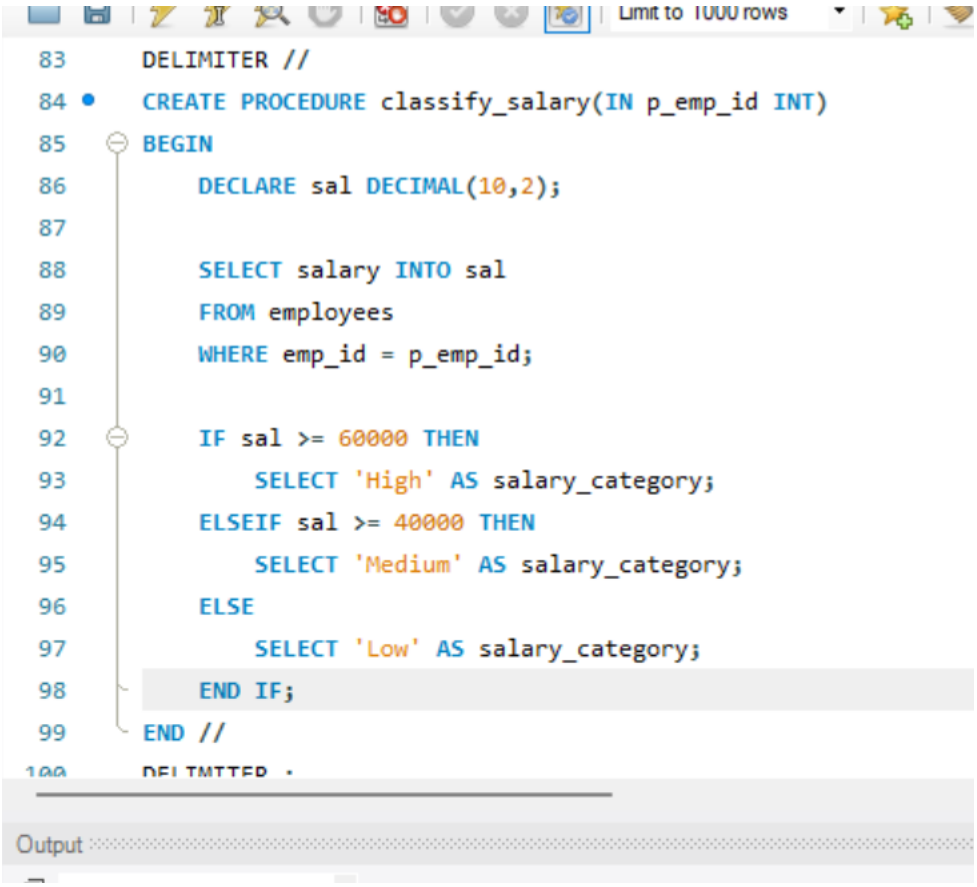
```
72    DELIMITER //
73    CREATE PROCEDURE high_avg_salary()
74    BEGIN
75        SELECT dept_id, AVG(salary) AS avg_salary
76        FROM employees
77        GROUP BY dept_id
78        HAVING AVG(salary) > 55000;
79    END //
80    DELIMITER ;
81    CALL high_avg_salary();
```

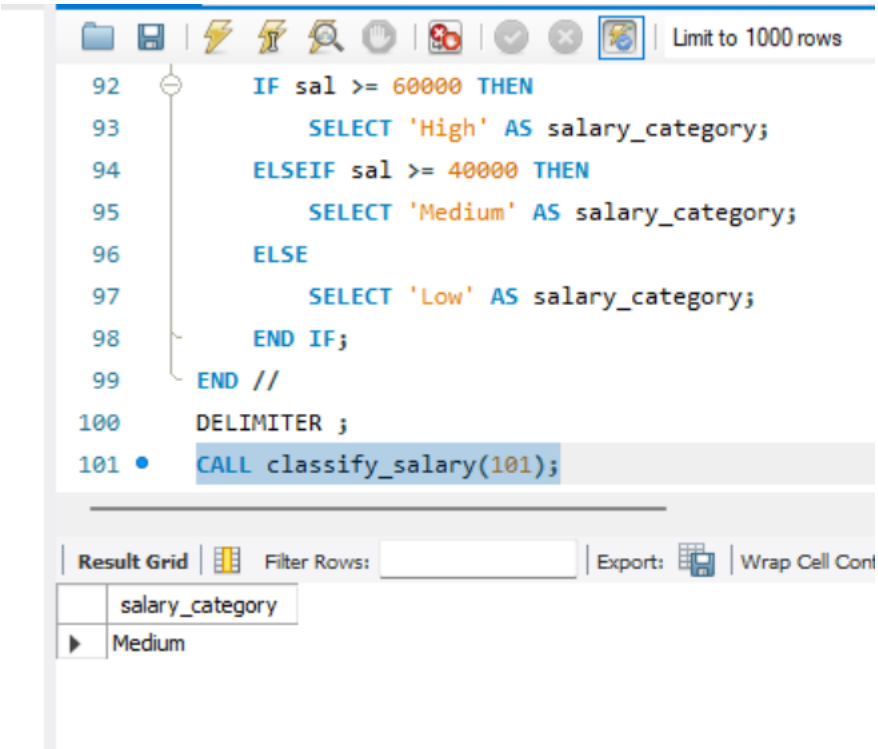
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_id	avg_salary
3	61500.000000
2	57500.000000

Result 9

7. Create a procedure using IF...ELSE to classify salary as High, Medium or Low.


Limit to 1000 rows
DELIMITER //
CREATE PROCEDURE classify_salary(IN p_emp_id INT)
BEGIN
DECLARE sal DECIMAL(10,2);
SELECT salary INTO sal
FROM employees
WHERE emp_id = p_emp_id;
IF sal >= 60000 THEN
SELECT 'High' AS salary_category;
ELSEIF sal >= 40000 THEN
SELECT 'Medium' AS salary_category;
ELSE
SELECT 'Low' AS salary_category;
END IF;
END //
Output


Limit to 1000 rows
IF sal >= 60000 THEN
SELECT 'High' AS salary_category;
ELSEIF sal >= 40000 THEN
SELECT 'Medium' AS salary_category;
ELSE
SELECT 'Low' AS salary_category;
END IF;
END //
DELIMITER ;
CALL classify_salary(101);
Result Grid
Filter Rows:
Export:
Wrap Cell Cont
salary_category
Medium

8. Create a procedure that accepts a department ID and returns its average salary.

```
101 •  CALL classify_salary(101);
102
103    DELIMITER //
104 •  CREATE PROCEDURE avg_salary(
105        IN p_dept_id INT,
106        OUT avg_sal DECIMAL(10,2)
107    )
108    BEGIN
109        SELECT AVG(salary)
110        INTO avg_sal
111        FROM employees
112        WHERE dept_id = p_dept_id;
113    END //
114    DELIMITER ;
115 •  CALL avg_salary(1, @avg);
116 •  SELECT @avg;
```

Output

ibles

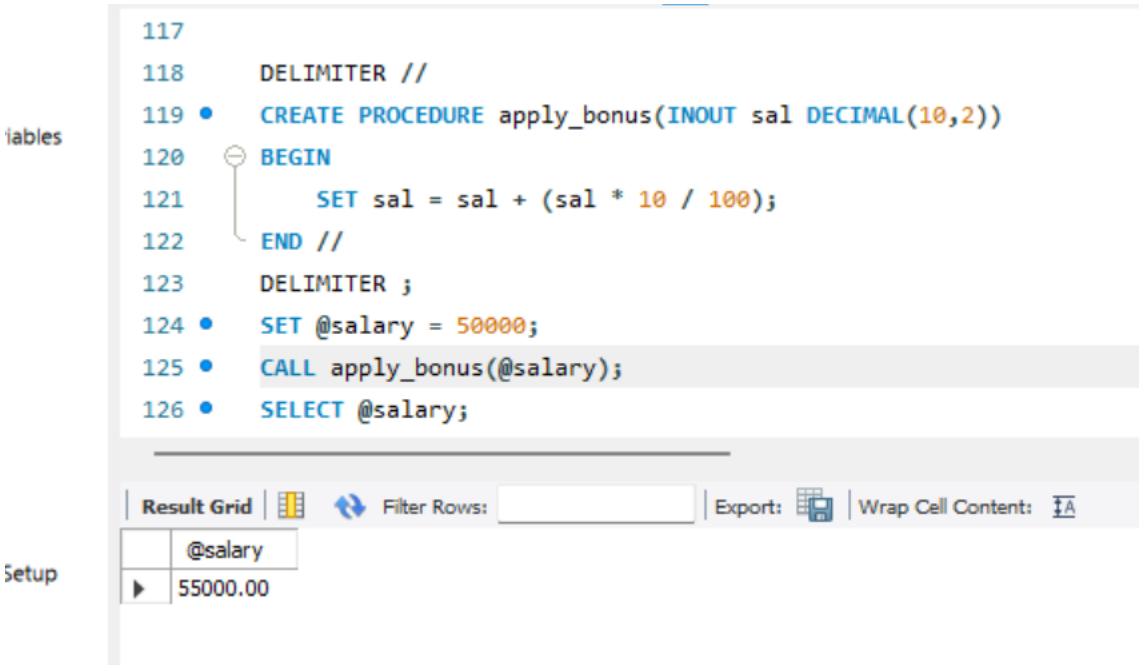
```
108    BEGIN
109        SELECT AVG(salary)
110        INTO avg_sal
111        FROM employees
112        WHERE dept_id = p_dept_id;
113    END //
114    DELIMITER ;
115 •  CALL avg_salary(1, @avg);
116 •  SELECT @avg;
```

Result Grid | Filter Rows: | Ex

	@avg
▶	55000.00

tup

9. Create a procedure using INOUT to apply a bonus to an input salary.

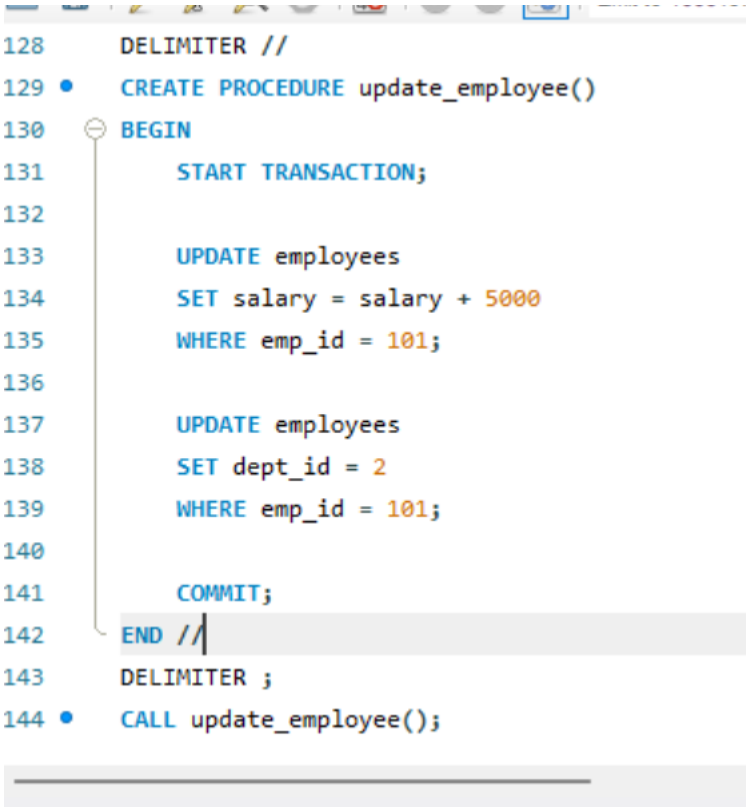


```
117
118     DELIMITER //
119     CREATE PROCEDURE apply_bonus(INOUT sal DECIMAL(10,2))
120     BEGIN
121         SET sal = sal + (sal * 10 / 100);
122     END //
123     DELIMITER ;
124     SET @salary = 50000;
125     CALL apply_bonus(@salary);
126     SELECT @salary;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

@salary
55000.00

10. Create a procedure that performs two related updates inside a transaction.



```
128 DELIMITER //
129 CREATE PROCEDURE update_employee()
130 BEGIN
131     START TRANSACTION;
132
133     UPDATE employees
134     SET salary = salary + 5000
135     WHERE emp_id = 101;
136
137     UPDATE employees
138     SET dept_id = 2
139     WHERE emp_id = 101;
140
141     COMMIT;
142 END //
143 DELIMITER ;
144 CALL update_employee();
```

```
137     UPDATE employees
138     SET dept_id = 2
139     WHERE emp_id = 101;
140
141     COMMIT;
142 END //
143 DELIMITER ;
144 CALL update_employee();
145 select * from employees;
146
```

Result Grid | Filter Rows: | Edit:

emp_id	emp_name	dept_id	salary
101	Rahul	2	58000.00
102	Priya	2	67000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00
105	Kiran	2	48000.00

employees 14 ×